UNIVERSIDAD DE CARTAGENA

FACULTAD DE INGENIERÍA

PROGRAMA DE INGENIERÍA DE SOFTWARE

CENTRO TUTORIAL CERETÉ

SEGURIDAD INFORMÁTICA

Endurecimiento de una aplicación de gestión documental

Auditoría, explotación y mitigación con evidencia reproducible:
un binario en Rust con dos políticas de seguridad

Trabajo Colaborativo Contextualizado (TCC)

PRESENTADO POR

Sergio Luis Doria Días

Juan David Cardona Bageth

Christian David Traeger Gamero

DOCENTE

Rafael Enrique Benedetti Navarro

SEMESTRE

2026-2

Cereté, Córdoba — 20 de noviembre de 2026

Contenido

1. Resumen ejecutivo 4

2. Introducción 4

2.1 Contexto 4

2.2 Aporte del trabajo 4

3. Planteamiento del problema 4

3.1 Problema 4

3.2 Justificación 5

3.3 Pregunta de investigación 5

4. Objetivos y alcance 5

4.1 Objetivo general 5

4.2 Objetivos específicos 5

4.3 Alcance 5

4.4 Limitaciones y exclusiones 6

5. Marco teórico 6

5.1 OWASP Top 10 (2021) 6

5.2 MAGERIT v3 6

5.3 Criptografía aplicada 6

5.4 STRIDE 7

5.5 Defensa en profundidad 7

6. Metodología 7

6.1 Diseño del experimento controlado 7

6.2 La suite de ataques como especificación ejecutable 7

6.3 Fases 7

6.4 Consideraciones éticas y legales 7

7. Diseño de la solución 8

7.1 Arquitectura del laboratorio 8

7.2 Flujo de datos y amenazas (STRIDE) 8

7.3 Análisis de riesgos (MAGERIT v3, escala 1–5) 8

7.4 Controles y contramedidas 9

8. Implementación 10

8.1 Tecnologías y decisiones 10

8.2 La aplicación en dos modos 10

8.3 Detalles de implementación relevantes 11

8.4 Panel de evidencias y demostración en vivo 11

9. Pruebas y resultados 12

9.1 Matriz antes / después 12
9.2 Hallazgos destacados 12
9.3 La suite detectó dos defectos propios 13
9.4 Evidencia visible por ataque 13
9.5 Evidencia disponible 13
10. Discusión 14
11. Conclusiones 14
12. Recomendaciones y trabajo futuro 15
13. Referencias 15
14. Anexos 16
Anexo A. Reproducción completa 16
Anexo B. Trazabilidad requisito → artefacto 16
Anexo C. Distribución de roles 16

1. Resumen ejecutivo

Este trabajo construye una aplicación web de gestión documental —**MiniGestor Documental**— implementada en Rust y ejecutada íntegramente en Docker, con **dos modos del mismo binario**: uno deliberadamente vulnerable y otro endurecido. El modo vulnerable materializa diez defectos de seguridad representativos del OWASP Top 10 2021; el modo endurecido aplica la contramedida correspondiente a cada uno. Sobre ambos corre una **suite de nueve ataques automatizados** que funciona como especificación ejecutable: cada ataque **debe** funcionar contra la versión vulnerable y **debe** quedar bloqueado en la versión endurecida.

Resultado verificado: **18 de 18 expectativas cumplidas** (nueve ataques por dos modos), sin fallos ni pruebas omitidas, regenerables con un único comando. La evidencia —respuestas HTTP crudas, salidas de cada prueba, capturas de tráfico y un panel interactivo que las presenta— queda disponible para revisión.

Los cuatro enfoques del curso están cubiertos: **OWASP Top 10**, **criptografía aplicada**, **análisis de riesgos MAGERIT v3** y **detección y simulación de ataques de red**. El aporte metodológico central es el uso de una **doble versión como experimento controlado**: la misma aplicación, el mismo esquema de datos y las mismas pruebas, cambiando únicamente las decisiones de seguridad. Así, la afirmación «el control funciona» deja de ser una opinión y se convierte en un resultado reproducible por un tercero.

2. Introducción

2.1 Contexto

Las organizaciones que digitalizan procesos administrativos acumulan documentos con información sensible —contratos, historias clínicas, datos financieros— en aplicaciones web construidas con plazos cortos y sin revisión de seguridad. El patrón de fallo es conocido: autenticación con funciones de hash obsoletas, autorización verificada solo en la interfaz, consultas SQL construidas por concatenación, archivos almacenados en claro y ausencia de trazabilidad. Cada defecto, por separado, parece menor; en conjunto producen brechas con impacto regulatorio.

2.2 Aporte del trabajo

La seguridad informática suele enseñarse como cuatro temas separados: listas de vulnerabilidades, algoritmos criptográficos, marcos de gestión de riesgos y herramientas de pentesting. En la práctica operan juntos: un defecto de validación de entrada se convierte en fuga de datos, la fuga permite romper un esquema de hashing débil, y el esquema débil anula cualquier control perimetral. Este trabajo integra los cuatro enfoques sobre **un solo sistema** para mostrar esas cadenas de dependencia, y agrega una decisión de ingeniería que lo hace verificable: la suite de ataques es también el criterio de aceptación del proyecto.

3. Planteamiento del problema

3.1 Problema

Una aplicación de gestión documental sin controles de seguridad permite a un atacante con conocimientos básicos leer documentos de otros usuarios, recuperar credenciales, escalar privilegios y escribir archivos arbitrarios en el servidor, sin dejar rastro auditable.

3.2 Justificación

- **Académica:** cubre los cuatro enfoques del curso en un único artefacto verificable, en lugar de cuatro actividades inconexas.
- **Profesional:** reproduce el ciclo real de trabajo —análisis de riesgos, prueba de penetración, remediación y re-verificación—, que es el que exigen marcos como ISO/IEC 27001 y OWASP ASVS.
- **Didáctica:** el modo vulnerable es material de estudio; el modo endurecido permite contrastar cada mitigación con su defecto en el mismo archivo de código.
- **Sostenibilidad:** el laboratorio completo se levanta con un comando, sin máquinas virtuales ni herramientas instaladas en el equipo, de modo que cualquier integrante o el docente reproduce los resultados.

3.3 Pregunta de investigación

¿Qué cambia exactamente entre una aplicación que un atacante compromete en minutos y la misma aplicación con controles aplicados? La respuesta se busca por comparación de ejecuciones —los mismos nueve ataques contra las dos versiones— y no por comparación de documentación.

4. Objetivos y alcance

4.1 Objetivo general

Implementar mecanismos de seguridad informática en una aplicación web de gestión documental, mediante análisis de riesgos, criptografía aplicada, controles del OWASP Top 10 y pruebas de penetración, demostrando el antes y el después con evidencia técnica reproducible.

4.2 Objetivos específicos

1. Diseñar la arquitectura del laboratorio y de la aplicación con dos modos de operación sobre un mismo código.
2. Identificar activos, amenazas y riesgos aplicando MAGERIT v3 en escala simplificada, con trazabilidad a controles concretos.
3. Implementar la versión vulnerable con diez defectos representativos de las categorías A01, A02, A03, A04, A05, A07 y A09 del OWASP Top 10 2021.
4. Implementar la versión endurecida con Argon2id, AES-256-GCM, sesiones firmadas con estado en servidor, segundo factor TOTP, control de acceso verificado en servidor, validación de entradas, cabeceras de seguridad y auditoría.
5. Construir una suite de ataques automatizada que verifique, para cada defecto, su explotación en la versión vulnerable y su bloqueo en la versión endurecida.
6. Producir evidencia verificable: respuestas HTTP, capturas de tráfico, resultados consolidados y un panel interactivo de consulta.
7. Documentar el proceso, las pruebas y los resultados de forma que un tercero pueda reproducirlos.

4.3 Alcance

La aplicación incluye autenticación, autorización, carga, búsqueda, lectura y descarga de documentos, panel administrativo y auditoría. El laboratorio incluye base de datos, gateway TLS, red aislada y herramientas de captura. Se cubren los cuatro enfoques del curso, más la suite de ataques y el panel de evidencias.

4.4 Limitaciones y exclusiones

- No se realizan pruebas contra sistemas reales de la universidad ni de terceros: todo el tráfico se genera dentro de la red del laboratorio.
- El análisis MAGERIT es una aplicación metodológica en escala 1–5; no incluye análisis de dependencias entre activos ni valoración económica.
- El firewall (nftables) se documenta como política de referencia ejecutada en un contenedor propio; no sustituye un cortafuegos perimetral real.
- La aplicación no implementa firma digital de documentos, análisis antimalware de archivos cargados ni federación de identidad: son trabajo futuro declarado.
- Siete de los nueve ataques se ejecutan desde el panel de evidencias; los dos que requieren privilegios del host (path traversal y captura de tráfico) se ejecutan con el comando que el panel indica.
- Los secretos incluidos (clave maestra, secreto de sesión, contraseñas demo) son **valores de laboratorio** y su exposición forma parte del ejercicio de auditoría.

5. Marco teórico

5.1 OWASP Top 10 (2021)

Catálogo de las categorías de riesgo más relevantes en aplicaciones web. Este trabajo aborda **A01** control de acceso quebrado, **A02** fallas criptográficas, **A03** inyección, **A04** diseño inseguro, **A05** configuración incorrecta, **A07** fallas de identificación y autenticación y **A09** fallas de registro y monitoreo.

5.2 MAGERIT v3

Metodología española de gestión de riesgos de sistemas de información. Modela **activos** —clasificados en dimensiones de confidencialidad, integridad, disponibilidad, autenticidad y trazabilidad—, **amenazas** con frecuencia estimada e **impacto** expresado como degradación del activo. El riesgo se aproxima como `riesgo = frecuencia × degradación`; sobre ese valor se seleccionan **salvaguardas** y se evalúa el **riesgo residual**.

5.3 Criptografía aplicada

- **Funciones de derivación de claves frente a funciones de hash simples.** MD5 fue diseñado para ser rápido, propiedad exactamente opuesta a la deseable para almacenar contraseñas. Argon2id (RFC 9106) incorpora costo de memoria, de tiempo y de paralelismo, además de una sal única por usuario.
- **Cifrado autenticado (AEAD).** AES-256-GCM garantiza confidencialidad e integridad en una sola operación; exige un nonce distinto por mensaje, condición que este trabajo cumple generando doce bytes aleatorios por documento.
- **Integridad de sesión.** HMAC-SHA256 sobre el identificador de sesión permite detectar cookies manipuladas, con comparación en tiempo constante para no filtrar información por temporización.
- **Segundo factor TOTP (RFC 6238).** Código de seis dígitos derivado de HMAC-SHA1 sobre un contador de treinta segundos, con ventana de tolerancia de ±1 paso.
- **Transporte.** TLS 1.2/1.3 con cifrados AEAD y HSTS; cookies `Secure` para que el navegador no las envíe por canal no cifrado.

5.4 STRIDE

Modelo de amenazas que clasifica los riesgos en **S**uplantación, **T**ampering, **R**epudio, **I**nformación expuesta, **D**enegación de servicio y **E**levación de privilegios. Se aplica sobre el diagrama de flujo de datos para ubicar cada amenaza en un flujo concreto.

5.5 Defensa en profundidad

Principio de superponer controles independientes para el mismo riesgo, de modo que el fallo de uno no implique el compromiso total. Aquí se aplica de forma verificable: el límite de intentos existe en la aplicación **y** en el gateway; el escape de salida convive con una política CSP; el cifrado en reposo convive con el cifrado en tránsito.

6. Metodología

6.1 Diseño del experimento controlado

La decisión metodológica central es **no comparar dos proyectos**, sino **un proyecto con dos políticas**. La aplicación es un único binario cuyo comportamiento depende de la variable de entorno `MODE` (`vulnerable` o `hardened`). Comparten esquema de base de datos, rutas, vistas y flujo; difieren en las decisiones de seguridad. Cada instancia usa además una base de datos independiente, de modo que las evidencias del «después» no se contaminan con el estado del «antes».

6.2 La suite de ataques como especificación ejecutable

Cada ataque es un script con un contrato explícito de salida: `0` cuando el ataque tuvo éxito (el defecto está presente), `1` cuando fue bloqueado (la mitigación funciona) y `2` cuando la prueba no pudo ejecutarse. El orquestador ejecuta los nueve ataques contra ambos modos, compara cada resultado con la expectativa y produce un archivo de resultados; si una expectativa falla, termina con código de error y el panel lo muestra en rojo.

Esta elección convierte la verificación en algo repetible y resistente al autoengaño: el informe no puede afirmar que una mitigación funciona si la suite no lo demuestra.

6.3 Fases

1. **Análisis** — inventario de activos, identificación de amenazas (STRIDE) y valoración de riesgos (MAGERIT).
2. **Construcción** — implementación de la versión vulnerable con los defectos trazados a riesgos.
3. **Explotación** — ejecución de la suite contra la versión vulnerable.
4. **Remediación** — implementación de los controles en la versión endurecida.
5. **Re-verificación** — misma suite contra la versión endurecida y análisis del riesgo residual.
6. **Documentación y evidencia** — consolidación de resultados y panel interactivo.

6.4 Consideraciones éticas y legales

Todo el tráfico se genera dentro de una red Docker aislada y los servicios se publican exclusivamente en `127.0.0.1`. Los datos son simulados. No se ejecuta ninguna prueba contra infraestructura de la universidad ni de terceros: hacerlo constituiría un delito conforme a la Ley 1273 de 2009, que tipifica el acceso abusivo a sistemas informáticos en Colombia.

7. Diseño de la solución

7.1 Arquitectura del laboratorio

La arquitectura se documenta en `docs/diagramas/arquitectura.svg`. Sus decisiones principales:

Componente	Función	Decisión de seguridad
`app-vuln` · :18080	Aplicación con defectos sembrados	Publicada solo en `127.0.0.1`, con aviso permanente
`gateway` · :18443	Terminación TLS, HSTS y límite de tasa	TLS 1.2/1.3, solo cifrados AEAD
`app-hard` · :18091	Aplicación endurecida, detrás del gateway	No publicada a Internet; en producción sería interna
`db` · PostgreSQL 16	Dos bases: `minigestor_vuln` y `minigestor_hard`	Sin puerto publicado: solo en la red del laboratorio
`attacker` · netshoot	`tcpdump` y utilidades de red	`NET_ADMIN` limitado al contenedor
`evidence` · :18090	Panel interactivo de evidencias	Solo lectura del repositorio; usuario sin privilegios
`lab_net` · 172.28.60.0/24	Red del laboratorio	Aísla el tráfico de las pruebas

7.2 Flujo de datos y amenazas (STRIDE)

El diagrama `docs/diagramas/dfd-stride.svg` identifica ocho flujos y las amenazas asociadas:

Flujo	Amenaza STRIDE	Defecto en la versión vulnerable	Ataque
F1	Suplantación	Sin límite de intentos; mensajes que enumeran cuentas	`04-bruteforce`
F2	Información expuesta	MD5 sin sal en la base de datos	`06-weak-hash`
F3	Tampering	Término de búsqueda concatenado en el SQL	`01-sqli-search`
F4	Elevación	Sin filtro por propietario en el resultado	`01-sqli-search`
F5	Tampering	Nombre del cliente usado como ruta de escritura	`07-traversal`
F6	Información expuesta	Documentos en claro con nombre predecible	`07-traversal`
F7	Elevación (cliente)	Salida sin escapar; cookie sin `HttpOnly`	`02-xss-stored`
F8	Repudio	Sin tabla de auditoría de eventos	`08-headers`

7.3 Análisis de riesgos (MAGERIT v3, escala 1–5)

El inventario completo está en `docs/riesgos/magerit.yml`, con doce riesgos valorados. Nivel: crítico ≥ 16, alto 9–15, medio 4–8, bajo ≤ 3.

ID	Activo	Tipo	Criticidad
A1	Servicio web de gestión documental	[S] servicio	Alta
A2	Base de datos: usuarios, metadatos, auditoría	[D] datos	Alta
A3	Documentos almacenados	[I] información	Alta
A4	Credenciales, secretos de sesión y clave maestra	[K] claves	Alta
A5	Red del laboratorio y gateway TLS	[COM] comunicaciones	Media
A6	Código fuente y artefactos de construcción	[SW] software	Media

Valoración. Riesgo = frecuencia × degradación.

ID	Amenaza	Activo	F	D	R	Nivel
R1	Inyección SQL en la búsqueda	A2	5	5	25	Crítico
R2	Lectura de documentos ajenos (IDOR)	A3	5	5	25	Crítico
R3	XSS almacenado y reflejado	A1	4	4	16	Crítico
R4	Recuperación de contraseñas por hash débil	A2	5	5	25	Crítico
R5	Falsificación de sesión y escalada	A4	5	5	25	Crítico
R6	Fuerza bruta contra el inicio de sesión	A1	5	4	20	Crítico
R7	Documentos expuestos en reposo	A3	4	5	20	Crítico
R8	Escritura arbitraria de archivos	A3	4	4	16	Crítico
R9	Secretos expuestos por endpoint de depuración	A4	3	5	15	Alto
R10	Interceptación de credenciales (MITM)	A5	4	5	20	Crítico
R11	Ausencia de trazabilidad	A1	4	3	12	Alto
R12	Configuración insegura del navegador	A6	3	3	9	Alto

Salvaguardas y riesgo residual.

ID	Salvaguarda aplicada en la versión endurecida	Residual
R1	Consultas parametrizadas con enlace de parámetros	Bajo
R2	Verificación de propiedad o rol en cada acceso, con auditoría	Bajo
R3	Escape de salida centralizado, CSP sin scripts, cookie `HttpOnly`	Bajo
R4	Argon2id (19 MiB, t=2, p=1) con sal única por usuario	Medio
R5	Identificador opaco + HMAC-SHA256 + tabla de sesiones con expiración	Bajo
R6	Cinco intentos por minuto, bloqueo temporal, TOTP y `limit_req`	Bajo
R7	AES-256-GCM con nonce aleatorio por documento	Bajo
R8	Lista blanca de extensiones, límite de 5 MiB, nombre opaco	Bajo
R9	El endpoint de depuración no existe en el binario endurecido	Bajo
R10	TLS 1.2/1.3, HSTS, redirección 301 y cookies `Secure`	Bajo
R11	Tabla `audit_log` con actor, acción, objetivo y resultado	Medio
R12	CSP, `X-Frame-Options`, `nosniff`, `Referrer-Policy`	Bajo

Los riesgos residuales R4 y R11 se asumen explícitamente: un hash Argon2id sigue siendo atacable con diccionario si la contraseña es débil —la mitigación real es la política de contraseñas y el segundo factor— y la auditoría local no sustituye un sistema centralizado con retención y alertas.

7.4 Controles y contramedidas

ID	Control implementado	Ubicación en el código
R1	Consultas parametrizadas	`routes/docs_routes.rs`
R2	Verificación de propiedad y rol	`routes/docs_routes.rs`

ID	Control implementado	Ubicación en el código
R3	Escape de salida y política CSP	`views.rs` , `security.rs`
R4	Argon2id con verificación de hashes legados	`auth.rs`
R5	Sesión en servidor firmada con HMAC	`session.rs`
R6	Limitador de intentos, bloqueo y TOTP	`security.rs` , `auth_routes.rs`
R7	Cifrado AES-256-GCM en reposo	`crypto.rs`
R8	Validación de carga y nombre opaco	`routes/docs_routes.rs`
R9	Endpoint de depuración solo en modo vulnerable	`routes/mod.rs`
R10	TLS, HSTS y cookies `Secure`	`lab/gateway/nginx.conf`
R11	Registro de auditoría y panel administrativo	`db.rs` , `admin_routes.rs`
R12	Middleware de cabeceras de seguridad	`security.rs`

8. Implementación

8.1 Tecnologías y decisiones

Capa	Tecnología	Justificación
Lenguaje	Rust (edición 2021)	Seguridad de memoria sin recolector y ecosistema criptográfico auditado
Servidor HTTP	`axum` 0.8 sobre `tokio`	Extracción de tipos explícita y composición por middleware
Acceso a datos	`sqlx` 0.8	Permite mostrar SQL concatenado y parametrizado en el mismo archivo
Base de datos	PostgreSQL 16	Restricciones, tipos y funciones de fecha; dos bases independientes
Contenedores	Docker 29.9 y Compose 5.6	Reproducibilidad sin instalar herramientas en el equipo
Gateway	nginx 1.27	Terminación TLS, HSTS y límite de tasa independiente
Panel	Sitio estático generado desde el repositorio	El panel lee las evidencias, no las produce

Tamaño del artefacto: **2.174 líneas** propias de Rust y HTML en `app/` , y **1.334 líneas** de shell, Python y HTML en `attacks/` , `site/` y el análisis de riesgos; **241 dependencias** fijadas en `Cargo.lock` para garantizar compilaciones reproducibles.

8.2 La aplicación en dos modos

El punto de decisión está centralizado, de modo que el contraste sea visible en el mismo archivo y a pocas líneas de distancia:

```
// v1 — DEFECTO A03: el término se concatena dentro del SQL.
let sql = format!(
    "SELECT ... FROM documents d JOIN users u ON u.id = d.owner_id \
     WHERE d.title LIKE '%{}%' ORDER BY d.id", term
);
sqlx::query(&sql).fetch_all(&state.pool).await?

// v2 — consulta parametrizada: el payload es texto, nunca sintaxis.
sqlx::query("SELECT ... WHERE d.title ILIKE $1 AND (d.owner_id = $2 OR $3) ...")
    .bind(format!("%{}%", term)).bind(user.id).bind(user.is_admin())
    .fetch_all(&state.pool).await?
```

El mismo patrón se repite en el almacenamiento de contraseñas, en el escape de salida, en la emisión de la sesión y en el cifrado de documentos.

8.3 Detalles de implementación relevantes

- **Cifrado en reposo.** Formato `[nonce 12 B][ciphertext + tag GCM]`. El nonce aleatorio por documento evita la reutilización, que en GCM es catastrófica.
- **Migración de hashes.** La verificación reconoce el formato PHC de Argon2id y, si el valor no lo es, cae al hash legado MD5. Eso permite migrar usuarios sin invalidar cuentas, requisito real de cualquier plan de remediación.
- **Segundo factor.** TOTP sobre HMAC-SHA1 con truncamiento dinámico (RFC 4226 §5.3), secreto de 160 bits en base32 (RFC 4648) y ventana de ±1 paso de treinta segundos.
- **Autorización.** En el modo endurecido el rol nunca se toma de la cookie: se consulta nuevamente en la base de datos en cada operación administrativa.
- **Auditoría.** Cada lectura, descarga, carga, inicio de sesión y acceso denegado genera un registro con actor, acción, objetivo y resultado.

8.4 Panel de evidencias y demostración en vivo

El panel (`127.0.0.1:18090`) abre con la presentación del proyecto y continúa con cuatro secciones demostrativas: matriz antes/después, análisis de riesgos con mapa de calor, diagramas y reproducción.

Incorpora un **ejecutor de pruebas**: cada fila de la matriz lanza el ataque real contra el modo elegido y devuelve la salida del script, el código de salida y la interpretación del resultado. La ejecución se sigue en vivo —fases de validación, ejecución, recolección e interpretación— y termina mostrando la **evidencia visible** extraída de la respuesta cruda: la tabla de usuarios y hashes que devolvió la inyección, el documento confidencial servido sin permiso con su contenido real, el fragmento de HTML que ejecuta el XSS, las contraseñas recuperadas por diccionario, los códigos HTTP de cada intento y el estado de cada cabecera de seguridad.

Honestidad del ritmo. Un ataque real tarda entre 50 y 300 ms. El panel separa las dos cosas: la marca de tiempo de cada línea es la real del servidor, la duración total medida se muestra al final y el retardo del revelado se declara como ritmo didáctico, ajustable en tres velocidades y con opción de saltarlo. Se gana legibilidad sin falsear ninguna medición.

Guardas del ejecutor. Un endpoint que lanza procesos es en sí mismo una superficie de ataque. La versión final se sometió a una verificación de seguridad independiente que confirmó la ausencia de ejecución arbitraria y de bypass de la lista blanca, y señaló debilidades de aislamiento y consumo que fueron corregidas antes de la entrega:

Hallazgo de la verificación	Corrección aplicada
Control de `Host` falsificable	Token por proceso servido sin CORS, enviado en `X-Panel-Token`
Salida y cuerpo sin límite	Cuerpo acotado a 4 KB y salida leída de archivo, recortada a 8 KB
Timeout sin cerrar procesos hijos	Grupo de procesos propio y terminación con `SIGKILL`
Caché compartido entre corridas	Caché de cookies por ejecución, dentro del directorio temporal
Proceso ejecutado como `root`	Usuario sin privilegios (uid 10001)
Fuga de rutas y trazas internas	Errores al registro del contenedor; mensaje genérico al cliente

9. Pruebas y resultados

9.1 Matriz antes / después

Ejecución verificada el 6 de noviembre de 2026, con salida consolidada en `evidence/results.json`.

Ataque	OWASP	Vulnerable	Endurecido
SQL injection en la búsqueda	A03	Éxito · HTTP 200	Bloqueado · 0 filas
XSS almacenado en el título	A03	Éxito · `<script>`	Bloqueado · escapado
IDOR en `/docs/{id}`	A01	Éxito · HTTP 200	Bloqueado · HTTP 403
Fuerza bruta en `/login`	A07	Éxito · 8 intentos	Bloqueado · HTTP 429
Falsificación de cookie	A01	Éxito · panel admin	Bloqueado · firma inválida
Hashes débiles	A02	Éxito · 3 claves	Bloqueado · Argon2id
Path traversal en la carga	A04	Éxito · escritura en `/tmp`	Bloqueado · nombre opaco
Cabeceras de seguridad	A05	Éxito · 4 ausentes	Bloqueado · 4 presentes
Credenciales en claro	A02	Éxito · legibles en `.pcap`	Bloqueado · solo cifrado

Resumen: 18 ejecuciones, 18 expectativas cumplidas, 0 fallos, 0 omitidas.

9.2 Hallazgos destacados

El hash es el eslabón más débil de la cadena (R4). Los hashes MD5 obtenidos por inyección se revierten con un diccionario de diez palabras: las tres cuentas demo se recuperan en menos de un segundo. Con Argon2id el mismo diccionario no produce coincidencias, y el costo de 19 MiB por intento es lo que hace inviable el ataque a escala.

Una cookie en texto plano equivale a no tener autenticación (R5). Enviar `Cookie: sid=alice:admin` abre el panel administrativo sin conocer ninguna contraseña. El defecto no está en la criptografía sino en el diseño: el servidor confía en un valor que el cliente controla.

La inyección SQL no necesita herramientas (R1). El payload `' UNION SELECT 1, username, password_hash, 'x', 0, 'y' FROM users --` devuelve los hashes en la misma tabla de resultados. Funciona con una sola línea de `curl`: no hace falta `sqlmap` para demostrarlo, y por eso «nadie nos va a atacar así» es una premisa falsa.

La captura de tráfico convierte una sospecha en prueba (R10). La captura `09-http.pcap` contiene la contraseña en texto claro; la captura equivalente sobre TLS no contiene ningún dato legible.

La auditoría es un control, no un requisito burocrático (R11). Sin registro, un IDOR exitoso es indistinguible de un acceso legítimo. Con auditoría, el intento queda registrado con actor, objetivo y resultado, y la detección pasa a ser posible.

9.3 La suite detectó dos defectos propios

La primera ejecución completa arrojó 16 de 18. Los dos fallos aparecieron en el modo vulnerable, es decir, donde los ataques deben funcionar: la suite se estaba engañando a sí misma. El análisis produjo dos correcciones de valor metodológico:

1. **Sentencia SQL en varias líneas.** El `--` del payload comentaba solo el final de su línea y el `ORDER BY` siguiente rompía la consulta, de modo que la inyección producía un error y no una fuga. Se reescribió en una sola línea, que es además como la construyen la mayoría de las aplicaciones reales.
2. **`curl -F` interpreta el carácter `<`.** El título malicioso se leyó como «tomar el valor desde un archivo». La solución fue `--form-string`, que trata el valor como literal.

Ninguna de las dos correcciones tocó el comportamiento de seguridad buscado. El resultado final, tras verificarlas, es 18 de 18.

9.4 Evidencia visible por ataque

El panel no solo muestran la salida del script: parsea la respuesta cruda que el ataque acaba de obtener y presenta el dato.

Ataque	Evidencia que se muestra en pantalla
IDOR	El documento confidencial servido, con propietario y contenido real
SQL injection	Tabla de usuarios y hashes filtrados, más el conteo de documentos ajenos
XSS almacenado	El fragmento de HTML que el navegador ejecuta, o su versión escapada
Hashes débiles	Usuario, hash almacenado y contraseña recuperada por diccionario
Fuerza bruta	Código HTTP de cada intento y si hubo bloqueo
Cookie falsificada	La tabla de usuarios del panel administrativo abierto sin clave
Cabeceras	Estado de cada cabecera de seguridad en la respuesta

9.5 Evidencia disponible

Evidencia	Ruta
Resultado consolidado de la suite	`evidence/results.json`
Salida literal de cada ataque	`evidence/http/<modo>/<ataque>.log`
Respuestas HTTP crudas	`evidence/http/<modo>/*.html`
Capturas de tráfico	`evidence/network/09-http.pcap`, `09-tls.pcap`
Panel interactivo	`http://127.0.0.1:18090`
Diagramas generados por script	`docs/diagramas/*.svg`
Inventario de riesgos	`docs/riesgos/magerit.yml`

10. Discusión

Sobre el orden de las mitigaciones. El análisis de riesgos y la explotación coincidieron en la prioridad: control de acceso antes que criptografía. Una aplicación con AES-256-GCM pero sin verificación de propiedad sigue entregando documentos ajenos, solo que cifrados en disco. La criptografía protege el activo cuando el control de acceso ya funciona.

Sobre lo que costó más de lo previsto. El segundo factor y el cifrado en reposo fueron mecánicos; lo que consumió tiempo fue la coherencia: que cada defecto tuviera su control y su prueba, y que las tres piezas apuntaran al mismo archivo y a la misma línea. Ese trabajo de trazabilidad es el que convierte un ejercicio en un informe defendible.

Sobre el uso de Rust. Aporta seguridad de memoria y un ecosistema criptográfico confiable, y el binario es notablemente rápido; sin embargo, **no hace segura a la aplicación**. El defecto más grave del trabajo —confiar en el rol que envía el cliente— es de diseño y habría sido idéntico en cualquier lenguaje.

Sobre los límites de la demostración. La suite prueba lo que la suite prueba: los nueve defectos sembrados y sus nueve controles. No certifica la ausencia de otras vulnerabilidades. Un pentest profesional complementaría esto con análisis de dependencias, revisión de configuración del sistema operativo, pruebas de concurrencia y fuzzing de las rutas de carga.

Sobre la reproducibilidad de los diagramas. Los diagramas no se dibujaron a mano: un generador calcula bandas, corredores y etiquetas, y un verificador comprueba la geometría resultante —texto fuera del lienzo, texto sobre texto, texto sobre una caja ajena y flechas que atraviesan cajas— devolviendo error si algo se superpone. Un diagrama que se puede regenerar y verificar es documentación mantenible; una imagen editada a mano deja de actualizarse en cuanto cambia el sistema.

11. Conclusiones

1. Es posible demostrar el efecto de un programa de endurecimiento **con evidencia reproducible** en lugar de afirmaciones: 18 de 18 expectativas cumplidas, regenerables con un comando.
2. Los cuatro enfoques del curso no son temas independientes: forman una cadena. MAGERIT prioriza, el pentesting verifica, la criptografía y los controles del OWASP Top 10 corrigen, y la auditoría sostiene la detección posterior.
3. El defecto más grave encontrado no fue criptográfico sino de **diseño de autorización**: confiar en datos del cliente. La criptografía no compensa un control de acceso ausente.
4. La construcción de una suite de ataques como especificación ejecutable **detectó dos errores en las propias pruebas**, lo que confirma que la verificación automatizada disciplinada —y no la revisión manual— sostiene la credibilidad de un informe de seguridad.
5. El laboratorio en contenedores elimina la barrera de infraestructura: cualquier integrante, o el docente, reproduce el 100 % de los resultados en un equipo de 8 GB de RAM sin instalar herramientas de seguridad.
6. La doble versión sobre un mismo código resultó el recurso didáctico más potente del trabajo: permite ver el defecto, la mitigación y el resultado de la prueba en el mismo archivo y en la misma sesión.

12. Recomendaciones y trabajo futuro

1. **Política de contraseñas y bloqueo progresivo.** El riesgo residual de R4 exige longitud mínima, verificación contra listas de contraseñas comprometidas y bloqueo con retroceso exponencial.
2. **Anti-repetición en TOTP.** Almacenar el último paso aceptado por usuario para impedir la reutilización de un código dentro de la ventana de tolerancia.
3. **CSRF explícito.** `SameSite=Lax` mitiga, pero un token sincronizador en formularios es la defensa completa.
4. **Limitador distribuido.** El límite actual vive en memoria del proceso; en un despliegue con varias instancias debe residir en un almacén compartido.
5. **Análisis de archivos cargados.** Verificación de tipo real por contenido y análisis antimalware antes de aceptar un documento.
6. **Rotación de sesión y de claves.** Rotar el identificador al autenticarse y al cambiar de rol; planificar la rotación de la clave maestra de documentos.
7. **Centralización de la auditoría.** Enviar los eventos a un colector con retención, correlación y alertas, y definir reglas de detección sobre los eventos que esta suite ya genera.
8. **Integración continua de la seguridad.** Ejecutar la suite en cada cambio del código como puerta de calidad y añadir análisis de dependencias al flujo.
9. **Firma digital de documentos.** Incorporar firmas con clave pública para garantizar autenticidad y no repudio del contenido almacenado.

13. Referencias

1. OWASP Foundation. *OWASP Top 10 — 2021*. https://owasp.org/Top10/
2. OWASP Foundation. *Application Security Verification Standard (ASVS) 4.0.3*.
3. Ministerio de Hacienda y Administraciones Públicas (España). *MAGERIT v3 — Metodología de Análisis y Gestión de Riesgos de los Sistemas de Información*.
4. ISO/IEC 27005:2022. *Guidance on managing information security risks*.
5. M'Raihi, D. et al. *TOTP: Time-Based One-Time Password Algorithm*. RFC 6238, IETF, 2011.
6. M'Raihi, D. et al. *HOTP: An HMAC-Based One-Time Password Algorithm*. RFC 4226, IETF, 2005.
7. Biryukov, A., Dinu, D., Khovratovich, D., Josefsson, S. *Argon2 Memory-Hard Function for Password Hashing*. RFC 9106, IETF, 2021.
8. Grassi, P. A. et al. *Digital Identity Guidelines: Authentication and Lifecycle Management*. NIST SP 800-63B, 2017.
9. NIST. *Transitioning the Use of Cryptographic Algorithms and Key Lengths*. SP 800-131A Rev. 2, 2019.
10. Shostack, A. *Threat Modeling: Designing for Security*. Wiley, 2014.
11. Congreso de la República de Colombia. *Ley 1273 de 2009*, delitos informáticos.
12. Rust Project. *The Rust Programming Language*; documentación de `axum`, `sqlx`, `argon2`, `aes-gcm`, `hmac`.
13. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*.

14. Anexos

Anexo A. Reproducción completa

```
git clone <repo> tcc-seguridad && cd tcc-seguridad
docker compose up -d --build           # base, app v1, app v2, gateway TLS y panel
bash attacks/run-all.sh                # 18 ejecuciones → evidence/results.json
docker compose restart evidence        # refresca el panel en :18090
```

Un ataque individual:

```
bash attacks/03-idor.sh http://127.0.0.1:18080 vulnerable   # 0: el defecto existe
bash attacks/03-idor.sh https://127.0.0.1:18443 hardened     # 1: bloqueado
```

Anexo B. Trazabilidad requisito → artefacto

Requisito de la actividad	Artefacto que lo cumple
Caso real o simulado	MiniGestor Documental (aplicación simulada en Rust)
Mínimo dos enfoques	Los cuatro: OWASP, criptografía, MAGERIT y red
Diagrama de arquitectura segura	`docs/diagramas/arquitectura.svg`
Flujo de datos y amenazas	`docs/diagramas/dfd-stride.svg` y tabla STRIDE (§7.2)
Controles y contramedidas	§7.4 y `docs/riesgos/magerit.yml`
Mecanismos de cifrado	AES-256-GCM, Argon2id, HMAC-SHA256, TOTP, TLS
Autenticación	Argon2id, segundo factor TOTP y sesión firmada
Firewall	`lab/firewall/lab.nft` (política de filtrado de referencia)
Pruebas de penetración	`attacks/` (nueve ataques automatizados)
Herramientas	Docker, curl, sqlmap, hydra, Burp/ZAP, tcpdump, PostgreSQL
Documentación de pruebas y resultados	Este informe y el panel de evidencias

Anexo C. Distribución de roles

Actividad	Sergio	Juan David	Christian
Análisis de riesgos MAGERIT	R	C	C
Arquitectura y diagramas	R	C	C
Modelado de amenazas STRIDE	R	C	C
Aplicación vulnerable (v1)	C	R	C
Aplicación endurecida (v2)	C	R	C
Suite de ataques	C	C	R
Capturas de tráfico y evidencias	I	C	R
Panel interactivo de evidencias	I	C	R
Informe final	R	C	C
Sustentación	C	C	R

R = responsable · C = colabora · I = informado